

Correos

Componentes necesarios

Para que una aplicación envíe correos a sus usuarios debemos definir que componentes son necesarios. Enumeramos los siguientes:

- **Servidor de correo:** responsable de gestionar el envío y recepción de los correos electrónicos. Podemos optar por instalar un servidor local o utilizar un servicio de terceros que se encargue de la gestión, como puede ser Mailgun, Mailchimp, etc. Los servidores de correo utilizan el protocolo SMTP (Simple Mail Transfer Protocol).
- **Cliente de correo:** aplicación que se conecta al servidor para enviar o recibir los correos. En nuestro contexto, el cliente será nuestra propia aplicación de Laravel. Para poder conectarse al servidor de correo, una aplicación debe contar con las credenciales de acceso correctas.
- **Correo:** el propio contenido que será enviado, que consta del emisor, un remitente y el mensaje.

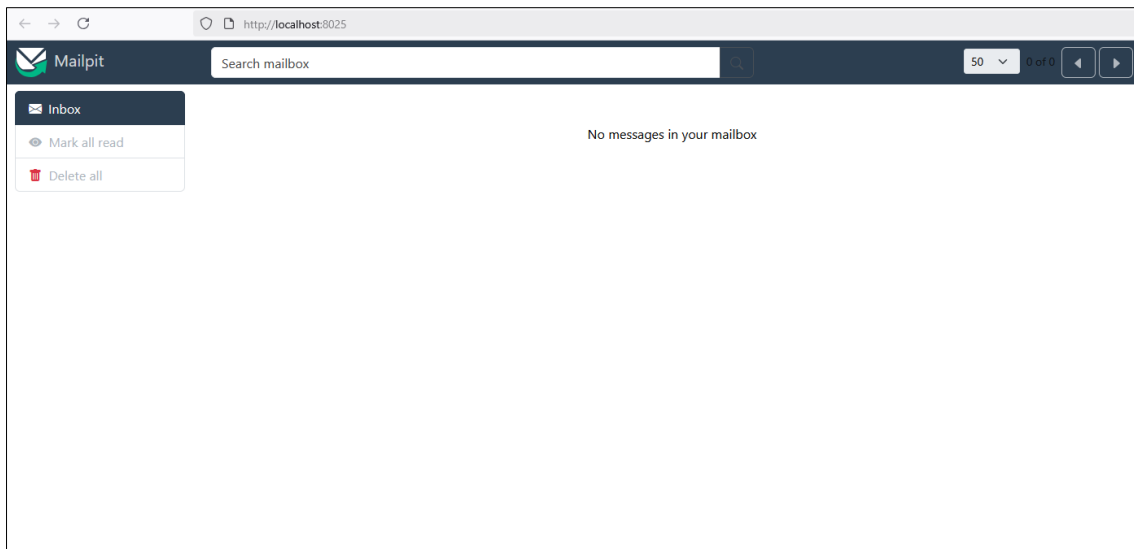
Servidor de correo

Para empezar a probar esta funcionalidad, instalaremos un servidor local conocido como Mailpit (<https://github.com/axllent/mailpit>), que nos permite disponer de un servidor con un comando y sin configuración necesaria.

Instalarlo es tan simple como ejecutar los siguientes pasos:

1. En la página de Github, nos vamos a la zona de lanzamientos ("*Releases*") y descargamos el ejecutable de nuestro sistema operativo.
2. Descomprimos el ejecutable en una carpeta que elijamos. Por ejemplo: en Windows podemos crear una carpeta en el disco C: llamada mailpit y ubicamos el archivo .exe allí dentro.
3. Luego abrimos una consola, navegamos a la ubicación de la carpeta y creada y simplemente ejecutamos el archivo ejecutable.

Una vez ejecutado, el servidor pondrá dos puertos a disposición: el puerto **1025** queda a disposición para recibir los correos desde nuestra aplicación; el puerto **8025** provee una interfaz web donde tendremos una bandeja de entrada para inspeccionar los correos recibidos.



Enviar correos con Laravel

Configuración

En el archivo **.env** debemos configurar los parámetros necesarios para poder enviar correos.

Variable	Descripción	Valor de ejemplo
MAIL_MAILER	Protocolo a utilizar	smtp
MAIL_HOST	Host del servidor de correo	localhost
MAIL_PORT	Puerto del servidor de correo	1025
MAIL_USERNAME	Nombre de usuario del servidor de correo	null
MAIL_PASSWORD	Contraseña del servidor de correo	null
MAIL_ENCRYPTION	Define si el servidor utiliza encriptación	null
MAIL_FROM_ADDRESS	Email del sitio que envía el correo	"soporte@ejemplo.com"
MAIL_FROM_NAME	Nombre del sitio que envía el correo	"\${APP_NAME}"

Mailables

Si bien es viable enviar correos desde una notificación, usando el método *toMail()*, para correos que tengan una complejidad mayor se prefiere el uso de los **Mailables** que podemos crear con Laravel.

Un **Mailable** es una clase que define los parámetros importantes de un correo, tales como:

- Sujeto y receptores del mail, mediante el método *envelope()*
- Contenido del mail, mediante el método *content()*
- Archivos adjuntos, mediante el método *attachments()*

Crear un Mailable

En consola ejecutamos el siguiente comando:

```
php artisan make:mail TransaccionCreadaMail
```

Esto creará el archivo en la carpeta **app/Mail**

Editar el contenido de la clase

Para configurar nuestro correo, abrimos el archivo **TransaccionCreadaMail.php** y editaremos cada uno de los bloques de importancia.

Primero, en el constructor de forma opcional podemos inyectar cualquier objeto sobre el cual necesitemos información. En este caso, inyectaremos una transacción para poder mostrar los datos de la transacción creada.

```
public function __construct(public Transaccion $transaccion)
{
    //
}
```

En el método `envelope()` modificaremos el sujeto del mail.

```
public function envelope(): Envelope
{
    return new Envelope(
        subject: 'Se ha creado una nueva transacción',
    );
}
```

Por último, en el método `content()` indicaremos que para enviar el mail se utilizará una vista que crearemos luego en la carpeta **resources/views/mail**.

```
public function content(): Content
{
    return new Content(
        view: 'mail.transaccion-creada',
    );
}
```

Crear la vista del correo

En la carpeta **resources/views** crearemos una nueva subcarpeta llamada **mail** (donde ubicaremos todas las vistas de los correos de nuestra aplicación) y dentro crearemos un nuevo archivo llamado **transacción-creada.blade.php**, con el siguiente contenido:

```
<h2>Nueva transaccion</h2>
```

<p>Se ha creado una nueva transaccion en nuestro sistema con los siguientes detalles:</p>

Descripcion: {{ \$transaccion->descripcion }}

Monto: {{ \$transaccion->monto }}

Fecha: {{ \$transaccion->fecha_transaccion }}

Como en el Mailable hemos inyectado los datos de la transacción, en la vista se pueden inspeccionar cada uno de los atributos de dicho objeto. Solamente nos resta enviar el mail con la transacción adecuada.

Enviar el mail

Para enviar el mail usamos la clase **Illuminate\Support\Facades\Mail**, la cual nos permite enviar un correo a un usuario puntual de la aplicación. En nuestro caso, editaremos el controlador de transacciones y agregaremos la siguiente línea luego del envío de la notificación:

```
Mail::to(auth()->user()->email)
    ->send(new TransaccionCreadaMail($transaccion));
```

Ahora podemos probar crear una transacción y recibiremos el mail en la bandeja de entrada de Mailpit. En caso de no recibirla o incurrir en un error, podemos comprobar que las credenciales están bien escritas, que el servidor de Mailpit esté funcionando o que no hayamos cometido algún error de tipeo en los pasos previos.

Rendimiento y Queue Workers

Si realizamos pruebas veremos que ahora el envío del formulario para crear la transacción es más lento que antes, porque hemos agregado un nuevo elemento de peso al controlador: la creación y el envío de un mail a un servicio externo. Al tener toda la lógica en el controlador, el usuario no será redirigido al index de transacciones hasta que el correo haya sido enviado al usuario. Esto es problemático por dos motivos principales:

1. Los procesos lentos irritan a los usuarios, hacen que pierdan la paciencia y no deseen utilizar nuestra aplicación. Esto es especialmente grave en sistemas que generan ganancias y tienen competencia, como es el mercado de los *eCommerce*. Cada segundo adicional es un potencial cliente perdido.
2. El envío del mail puede fallar (por ejemplo: si el servidor de correo que recibe el mail está caído momentáneamente o hay un micro corte de internet). Adicionalmente, la

aplicación puede reintentar enviar el correo por un determinado período de segundos, lo que aumentará aun más el tiempo de espera.

Para solucionar esta problemática, Laravel sugiere el uso de Queue Workers y Jobs para una aplicación web.

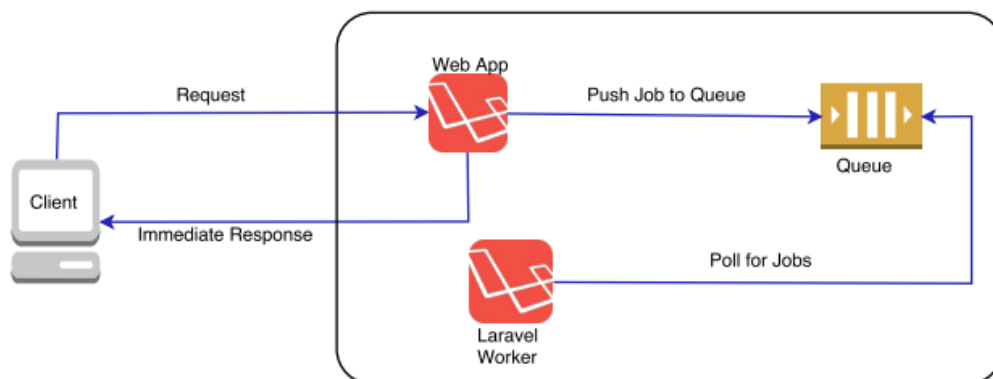
Queue Workers

Son procesos de PHP, separados del proceso principal de la aplicación, encargados de gestionar trabajos en una cola de espera y procesarlos a medida que van ingresando a la misma, quitándole la responsabilidad al hilo principal.

Consta de los siguientes 3 elementos:

- **Queue:** es la cola de espera que recibe los trabajos a ser procesados.
- **Jobs:** son los trabajos/tareas que son enviados desde la aplicación para ser procesados de forma asíncrona.
- **Worker:** es el proceso encargado de leer los trabajos agregados al queue y procesarlos.

Laravel implementa la arquitectura de los Queue Workers de la siguiente forma:



1. Un cliente (un usuario final) hace una petición a la aplicación web.
2. Si la aplicación web debe ejecutar un proceso que demanda tiempo, puede elegir delegar el trabajo (**Job**) a la cola de ejecución secundaria (**Queue**).
3. El cliente recibe una respuesta inmediata de la petición web, sin tener que esperar a que el trabajo se ejecute completamente.
4. El **Worker** consulta al **Queue** si existen trabajos disponibles que no hayan sido procesados. Si existen, los procesará y quitará de la cola de trabajo.

Tablas

Si elegimos trabajar con un queue en nuestra base de datos, tenemos que conocer las siguientes tablas:

- **jobs:** contiene todos los trabajos que son agregados al queue.
- **job_batches:** si los trabajos se agregan por lotes, esta tabla define cada lote y los trabajos que contiene.
- **failed_jobs:** en el caso de que un trabajo falle por cualquier motivo, se agrega a esta tabla, la cual luego puede ser consultada.

¿Qué tipo de Queue Worker necesitamos?

Para ejecutar un worker podemos optar por distintas soluciones, las cuales agruparemos de dos formas:

- Podemos trabajar con nuestra base de datos y un proceso de PHP local. Esta aproximación es la que usaremos como ejemplo.
- Podemos utilizar un servicio de terceros. Estos servicios pueden llegar a ser de pagos y algunas de ellos suelen tener un grado de complejidad intermedio, pero existen drivers que nos permitirán conectarlos a nuestra aplicación de Laravel. Algunos ejemplos populares son los siguientes:
 - **Redis:** <https://redis.io/>
 - **Amazon SQS:** <https://aws.amazon.com/es/sqs/>
 - **Beanstalkd:** <https://beanstalkd.github.io/>

Ejemplo: Queue worker basado en la base de datos

Para nuestro ejemplo, modificaremos el archivo **.env** para indicarle que utilizaremos la base de datos para ejecutar los Jobs:

```
QUEUE_CONNECTION=database
```

Luego, en una nueva consola levantaremos el proceso del Queue Worker de Laravel con el siguiente comando:

```
php artisan queue:work
```

Eso es todo, ahora dicho proceso estará monitoreando cuando ingrese un nuevo trabajo para poder ser procesado.

Pero, ¿cómo enviamos Jobs al Queue desde la aplicación? Exploreemos tres formas diferentes, según el caso que sea necesario.

Opción 1: método `queue()`

Si estamos trabajando con procesos del estilo notificaciones, mails u otros, podemos enviar directamente el proceso al queue mediante el método **`queue()`**.

```
Mail::to(auth()->user()->email)
    ->queue(new TransaccionCreadaMail($transaccion));
```

Opción 2: función dispatch()

Si queremos enviar manualmente un proceso al queue, incluso aquellos que no tienen la habilidad nativa (como mails o notificaciones) podemos usar la función **dispatch()** y definir una función anónima que defina lo que será procesado por el queue. Opcionalmente podemos agregar un retraso de X cantidad de segundos con el método delay().

```
dispatch(function() {
    logger("Enviando mensaje mediante un queue");
})->delay(5);
```

Opción 3: creando un Job con Laravel

La tercera opción incluye la creación de una clase Job en nuestra aplicación. Esto puede ser una ventaja para aquellos trabajos que requieren de una estructura definida o se repiten periódicamente en la aplicación.

Para crear la clase Job usamos el siguiente comando:

```
php artisan make:job TransaccionCreada
```

Esto creará la clase en la carpeta **app/Jobs**.

Editaremos la clase y agregaremos al constructor los datos que necesitamos para el trabajo en particular. En nuestro caso, el trabajo consiste en enviar un mail a una dirección de correo y con los datos de una transacción, por ello el constructor los definiremos de la siguiente forma:

```
public function __construct(
    public string $email,
    public Transaccion $transaccion)
{}
```

En el método **handle()** indicaremos qué debe ocurrir cuando se procesa este trabajo. Para el ejemplo actual solamente necesitamos enviar el correo.

```
public function handle(): void
{
    Mail::to($this->email)
        ->queue(new TransaccionCreadaMail($this->transaccion));
```

```
}
```

Por último, para despachar el trabajo se utiliza el método `dispatch()` junto con los parámetros que hemos declarado en nuestro constructor. Para ello, en el controlador remplazaremos la línea que enviaba el mail por la siguiente:

```
TransaccionCreada::dispatch(auth()->user()->email, $transaccion);
```

Una vez encendido el Queue Worker desde la consola, podremos ver que se ha agregado el trabajo a la **tabla jobs** en la base de datos y que luego será procesado y enviado a Mailpit. Si el proceso resulta muy rápido, podemos agregar el `delay()` a la línea de código anterior para poder retrasar el envío del mail.

Sugerencias para trabajar con correos

Seguridad

- **Evita la inyección de código:** Nunca inyectes directamente datos de usuario sin sanitizarlos. Blade permite escapar las variables con la sintaxis `{{ $variable }}`.
- **Limita la información sensible:** No incluyas datos confidenciales (como contraseñas o números de tarjetas de crédito) en los correos. Si necesitas que el usuario acceda a una sección privada, usa **tokens de un solo uso** con una URL firmada por Laravel (`URL::signedRoute()`) que expira después de un tiempo.
- **Usa un servicio de correo de terceros:** Para entornos de producción, no confíes en el servidor de correo de tu hosting. Servicios como Mailgun, Postmark o SendGrid ofrecen mayor fiabilidad, mejores tasas de entrega y funciones de seguridad avanzadas como **SPF** y **DKIM** que verifican la autenticidad de tus correos y evitan que sean marcados como spam.

Accesibilidad

- **Usa texto alternativo en las imágenes:** Describe el contenido de las imágenes para los usuarios que no pueden verlas o tienen las imágenes deshabilitadas en su cliente de correo.

```

```
- **Contraste de color adecuado:** Asegúrate de que el color del texto tenga un contraste suficiente con el color de fondo para que sea fácil de leer, especialmente para personas con deficiencias visuales.
- **Usa una estructura de HTML semántica:** Aunque el soporte es limitado en algunos clientes, usa encabezados (`<h1>`, `<h2>`), párrafos (`<p>`) y listas (`<ul>`, `<ol>`) para estructurar el contenido de manera lógica.

- **Simplifica el diseño:** Evita diseños complejos o con muchas columnas que puedan ser difíciles de leer en dispositivos móviles o con lectores de pantalla.

Compatibilidad

- **Usa tablas para el diseño:** Los correos electrónicos no son como las páginas web. La forma más segura de construir un diseño que se vea igual en la mayoría de los clientes de correo (incluyendo los más antiguos como Outlook) es usando **HTML anidado en tablas**. Evita el uso de Flexbox o Grid.
- **Estilos en línea (inline):** La mayoría de los clientes de correo no soportan las hojas de estilo externas (<link>) ni los bloques de estilo en el <head>. Usa el atributo style directamente en cada elemento. Por ejemplo:

```
<p style="font-family: Arial, sans-serif; font-size: 16px; color: #333;">
```

Texto del correo.

```
</p>
```

- **Usa una herramienta para verificar la compatibilidad:** en nuestro caso, **Mailpit** provee una pestaña que permite calcular la compatibilidad de las etiquetas usadas. Usa esta u otras herramientas que te permitan calcular si tu correo se verá bien en los clientes de correo más populares.

